

RazorRecall — End-to-End System Audit Report

Automated Payment Failure Recovery & Deterministic Ingestion Pipeline

Project: RazorRecall (Razorpay Buildathon 2026)	Runtime: Java 21, Spring Boot 4.1.1
Branch: main (Commit 50b07b6)	Database: PostgreSQL 18.6 (Flyway V1–V6 Frozen)
Audit Status: APPROVED & VERIFIED (8/8 Tests Passing)	Date: September 01, 2026

1. Executive Summary & Test Results

An end-to-end architectural and functional audit of the RazorRecall payment failure ingestion pipeline was conducted. The system processes incoming Razorpay payment.failed webhooks, performs signature and structural parsing, enforces idempotency, deterministically classifies failures (SOFT vs HARD), resolves merchant foreign keys, and establishes verified RecoveryCase records in PostgreSQL within transactional boundaries. All 8 automated tests passed with zero failures and zero schema regressions.

Test Suite / Class	Tests Run	Pass / Fail	Verified Capability
RazorrecallApplicationTests	1	1 / 0 (PASS)	Spring ApplicationContext, HikariCP, Flyway V1-V6 migration validation
FailureClassifierTest	3	3 / 0 (PASS)	Deterministic error categorization (SOFT=eligible, HARD=ineligible, UNKNOWN)
WebhookControllerTest	4	4 / 0 (PASS)	Nested/flat webhook ingestion, duplicate idempotency, bad payload handling

2. End-to-End Pipeline Execution Trace

- **Step 1: Ingestion Endpoint:** WebhookController.java:22 receives POST /api/webhooks/razorpay. Catches parsing/argument errors to return HTTP 400 Bad Request on malformed inputs and HTTP 200 on success.
- **Step 2: Transaction Orchestrator:** WebhookProcessorService.java:51 runs inside @Transactional. Coordinates payload extraction, deduplication, merchant resolution, attempt recording, and case creation atomically.
- **Step 3: Robust Payload Parser:** RazorpayWebhookPayload.java:30 extracts event type, payment ID, order ID, amount (paise to decimal), currency, error code, error description, and merchant/account ID from nested payload.payment.entity or flat JSON.
- **Step 4: Deduplication Gate:** Deduplicates on eventKey = eventType:paymentId via WebhookEventRepository.java:11. If already processed, immediately returns HTTP 200 with DUPLICATE status.
- **Step 5: Event Persistence:** WebhookEventService.java:16 saves raw payload and event metadata into webhook_events table.
- **Step 6: Merchant Resolution:** MerchantService.java:35 resolves merchant UUID deterministically (using RFC 4122 Type-3 name UUID for string IDs like acc_xxx), completely eliminating arbitrary fallbacks and guaranteeing foreign key constraints.
- **Step 7: Payment Attempt Persistence:** PaymentAttemptService.java:39 inserts or updates payment_attempts row with status = 'FAILED', amount, currency, and failure reason.
- **Step 8: Deterministic Classification:** FailureClassifier.java:48 evaluates error code and reason. Transient network/gateway/timeout issues yield SOFT (eligible=true); card expiry/fraud/insufficient funds yield HARD (eligible=false).

- **Step 9: Recovery Case Establishment:** RecoveryCaseService.java:45 creates recovery_cases record in DETECTED status with eligibility, classification, and nextActionAt = now + 5 min.

3. Detailed Verification Checklist

Verification Item	Target Behavior	Audit Status
1. Valid payment.failed webhook	Extracts nested payload.payment.entity, records failure attempt and recovery case.	VERIFIED
2. SOFT failure → Eligible	Codes like GATEWAY_ERROR, TIMEOUT yield SOFT, eligible=true, nextActionAt != null.	VERIFIED
3. HARD failure → Ineligible	Codes like CARD_EXPIRED, FRAUD yield HARD, eligible=false, nextActionAt = null.	VERIFIED
4. Duplicate Webhook / Idempotency	Subsequent identical webhooks return HTTP 200 DUPLICATE without inserting duplicate DB rows.	VERIFIED
5. Invalid / Malformed Webhook	Missing event type, missing payment ID, empty body, or invalid JSON returns HTTP 400 Bad Request.	VERIFIED
6. Merchant Resolution	Deterministic RFC 4122 Type-3 name UUID mapping satisfies payment_attempts.merchant_id FK constraint.	VERIFIED
7. Database Persistence & States	PostgreSQL rows saved with status FAILED (PaymentAttempt) and DETECTED (RecoveryCase).	VERIFIED

4. Architectural Risks & Observations

- **Concurrent Duplicate Race Condition:** If two identical webhooks hit at the exact same millisecond, both could pass application-level findByEventKey. The second thread triggers DB unique index constraint ux_webhook_events_event_key throwing DataIntegrityViolationException, which is currently caught by the generic catch block as HTTP 400 instead of HTTP 200 duplicate.
- **String vs Enum in Entity Mappings:** RecoveryCase.java and PaymentAttempt.java store status and failureClass as String to align with V1-V3 schema. While enums RecoveryStatus and FailureClass exist, type-safe enum converter or validation layer should guard against arbitrary strings in future blocks.
- **Paise vs Major Currency Amount Semantics:** RazorpayWebhookPayload divides integer amounts by 100 (paise to INR). For custom payloads passing pre-divided decimals, it preserves scale 2. This is robust for current tests but should be explicitly formalized in merchant configuration.

5. Downstream Scope for Subsequent Blocks

- **Block-2: Cryptographic Verification & Ingestion Hardening:** Add HMAC-SHA256 signature verification for header X-Razorpay-Signature using merchant webhook secret.
- **Block-3: Recovery Decision Engine & AI Boundary:** Integrate AI diagnosis sidecar (Python/FastAPI) to propose recovery strategies while backend deterministic guardrails validate before action dispatch.
- **Block-4: Action Dispatcher & Execution:** Implement customer communication, smart retry payment link generation, and terminal status polling.
- **Block-5: Lifecycle Completion & Recovery Metrics:** Ingest payment.captured webhooks to transition cases to RECOVERED and track recovered revenue metrics.

6. File & Code Traceability Matrix

Component	File Path	Lines	Responsibility
Controller	WebhookController.java	L22–L34	Exposes POST /api/webhooks/razorpay, returns HTTP 200/400
Processor	WebhookProcessorService.java	L51–L129	@Transactional orchestrator for deduplication & ingestion
DTO Parser	RazorpayWebhookPayload.java	L30–L141	Parses nested/flat Razorpay JSON payload into structured DTO
Classifier	FailureClassifier.java	L48–L96	Deterministic failure classification (SOFT/HARD/UNKNOWN + eligibility)
Merchant Svc	MerchantService.java	L24–L56	Deterministic merchant resolution & default merchant management
Payment Svc	PaymentAttemptService.java	L31–L71	Failed payment attempt persistence & lookup
Recovery Svc	RecoveryCaseService.java	L37–L73	Recovery case creation in DETECTED status with eligibility
Event Svc	WebhookEventService.java	L16–L25	Webhook event persistence & key uniqueness check